

LLMSheriff: A Hybrid Framework for Intent-State Inference in Autonomous AI Agents

Rajendra Kumar Shaw
Kolkata, India
rajendra250601@gmail.com

Abstract—Autonomous AI agents increasingly execute long-running workflows where failures are not always explicit. Existing observability platforms capture execution traces but generally do not infer whether an agent is still making meaningful progress toward its assigned goal. This paper presents LLMSheriff, an exploratory research prototype that investigates hybrid intent-state inference from agent execution traces. The system maps low-level events to eight behavioral states—Planning, Executing, Waiting, Recovering, Stalled, Abandoned, Completed, and Failed—using a rule-based analyzer and an LLM behavioral analyzer in parallel. Agreement between analyzers is treated as a confidence signal; disagreement is surfaced as a diagnostic output rather than suppressed. A behavior interpretation layer produces evidence summaries, confidence estimates, disagreement explanations, and human-intervention recommendations. We evaluate the rule-based analyzer on a corpus of 100 synthetic labelled traces and report 71% accuracy and 0.732 macro-F1, with strong performance on unambiguous cases and systematic errors on waiting and borderline traces. The work is exploratory; it does not claim to solve intent monitoring, but demonstrates one reproducible approach to behavioral inference above conventional observability.

Index Terms—agentic AI, execution monitoring, intent inference, goal satisfaction, behavioral observability

I. INTRODUCTION

Autonomous AI agents now orchestrate multi-step workflows involving tool calls, retries, external dependencies, and long idle periods. Standard observability outputs—logs, latency metrics, retry counts, and exit codes—record *what* happened during execution. They rarely answer whether the agent is still intentionally pursuing its assigned goal.

This distinction matters in practice. An agent may run without errors while making no forward progress: repeating the same search, polling indefinitely, or drifting after an early failure. Conversely, an agent may encounter setbacks and still recover by adopting a new strategy. A monitoring layer that only detects hard failures cannot separate these cases.

LLMSheriff is a research prototype built to explore this gap. It asks:

Can execution traces be interpreted to infer higher-level behavioral states that help a human decide whether to wait, intervene, or declare success?

The contribution is not a production monitoring platform. Rather, we investigate whether hybrid symbolic and LLM-based inference can produce explainable behavioral states from traces. A key design choice is that LLMSheriff does not attempt to infer an agent’s true internal intentions. Instead,

it infers *behavioral intent* from externally observable execution traces—a weaker but tractable formulation. When both analyzers agree, agreement is treated as a confidence signal. When they disagree, the disagreement itself is surfaced as a diagnostic output, identifying traces where semantic ambiguity requires human judgement.

II. BACKGROUND AND RELATED WORK

A. AI Observability and Agent Monitoring

Platforms such as LangSmith [6], Langfuse [7], and AgentOps provide trace storage, latency analysis, token accounting, and debugging views for LLM applications. OpenTelemetry [5] standardises distributed tracing across services. These tools excel at reconstructing execution paths and surfacing failures. They do not, by default, infer goal-level behavioral intent.

Recent work on LLM agent oversight focuses on detecting unsafe tool calls, hallucinated actions, and policy violations during execution. LLMSheriff differs by targeting *progress semantics*: whether execution still appears goal-directed rather than merely error-free. This is a softer but practically important property—an agent that loops safely is still failing its user.

B. Goal and Plan Recognition

Goal-oriented requirements engineering treats systems in terms of goals to be achieved and obstacles to satisfaction [1], [2]. In AI, plan and goal recognition infer an actor’s intentions from observed behaviour [3]. LLMSheriff applies a lightweight version of this idea to agent traces: it does not reconstruct full plans, but classifies coarse behavioral states that approximate human monitoring judgements.

C. Runtime Verification

Runtime verification checks whether system execution satisfies formal properties during operation [4]. LLMSheriff is less formal: it uses heuristic features and learned LLM judgement rather than temporal logic specifications. The motivation is similar—detect when execution diverges from expected progress—but the method is deliberately pragmatic for early-stage agent workflows.

D. Positioning

LLMSheriff does not replace existing observability stacks. It sits above them as an experimental behavioural interpretation layer, closer to goal-satisfaction monitoring than to trace visualisation.

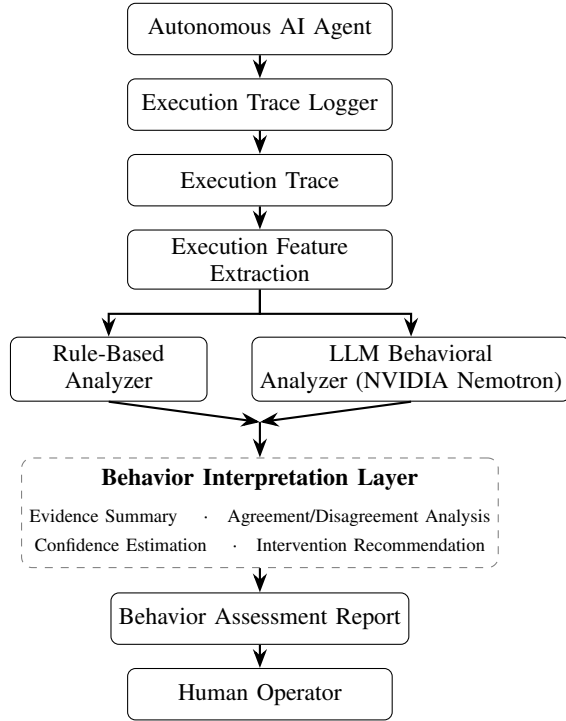


Fig. 1. LLMSheriff end-to-end pipeline. An autonomous agent produces an execution trace that undergoes feature extraction and parallel analysis by a rule-based engine and an LLM behavioral analyzer. The behavior interpretation layer aggregates their outputs—treating agreement as a confidence signal and disagreement as diagnostic evidence—into a structured behavior assessment report for a human operator.

III. MOTIVATION

While building a multi-agent workflow system, I observed a recurring failure mode: actors would satisfy local execution conditions while the higher-level task remained incomplete. Exit codes were clean, retries were within range, and logs showed activity, yet the overall goal was not advancing.

A related pattern appeared in my earlier work on automated answer evaluation: models could optimise internal similarity metrics while producing evaluations that were wrong from a human perspective. The system appeared finished; the task was not. LLMSheriff is motivated by the same structural problem at the agent-workflow level.

IV. SYSTEM DESIGN

A. Architecture

Figure 1 shows the end-to-end pipeline. Execution traces enter an execution feature-extraction stage, pass through two parallel analyzers, and feed a behavior interpretation layer that produces a structured assessment report for human operators.

B. Execution Feature Extraction

Each trace event contains a timestamp, step label, action string, duration, and status. After normalisation, eight features are computed: runtime, average latency, LLM call count, tool call count, retry count, failure count, longest streak of repeated

identical actions, and a progress score (success rate penalised for repetition, capped at 1.0).

C. Why a Hybrid Design?

Rule-based analyzer. Rules are fast, deterministic, and fully explainable. Thresholds can be inspected and modified without retraining. They perform well when feature patterns are unambiguous—multiple failures, extreme repetition with low progress, or clear recovery after errors.

LLM-based analyzer. Rules struggle on semantically ambiguous traces: polling that resembles healthy execution, loops with moderate progress scores, or abandonment without hard failures. The LLM behavioral analyzer receives the full trace and can reason about action semantics, stage transitions, and whether observed activity plausibly advances the goal. The trade-off is latency, API dependence, and less predictable behaviour across prompts.

Combining both. Agreement suggests a higher-confidence classification. Disagreement flags traces where features alone are insufficient—precisely the cases where semantic reasoning may help and where human review is most valuable.

D. State Vocabulary

The prototype uses eight states: Planning, Executing, Waiting, Recovering, Stalled, Abandoned, Completed, and Failed. The *Recovering* state is central to the research question: it represents an agent that encountered setbacks but adopted a new strategy rather than abandoning the goal.

E. Behavior Interpretation Layer

Beyond state labels, the behavior interpretation layer produces:

- **Evidence summary**—positive and negative trace-derived indicators;
- **Agreement/disagreement analysis**—when analyzers concur, agreement strengthens confidence; when they differ, disagreement is surfaced as a diagnostic signal;
- **Confidence estimation**—agreement-weighted confidence from both analyzers;
- **Intervention recommendation**—no action, continue monitoring, or human intervention suggested.

The aggregated output forms a *behavior assessment report*. The prototype dashboard visualises this report before it reaches a human operator.

V. IMPLEMENTATION

The backend is implemented in FastAPI with SQLite persistence. The frontend is a Next.js dashboard that visualises the behavior assessment report: dual predictions, metrics, an intent timeline, a behavior graph, evidence, disagreement explanations, and intervention recommendations. Six preset scenarios and a step-by-step replay demo support interactive exploration. The system runs via Docker Compose or local scripts.

The LLM behavioral analyzer calls NVIDIA Nemotron [8] through a chat-completions API with structured JSON output validation and heuristic fallback when the API is unavailable.

TABLE I
RULE-ENGINE PERFORMANCE ON 100 SYNTHETIC TRACES

Class	Prec.	Recall	F1	n
Completed	1.00	1.00	1.00	20
Recovering	1.00	1.00	1.00	20
Waiting	0.00	0.00	0.00	20
Stalled	0.79	0.75	0.77	20
Abandoned	1.00	0.80	0.89	20
Overall	Accuracy 0.71 / Macro-F1 0.732			100

TABLE II
SELECTED CONFUSION PATTERNS (GROUND TRUTH $\rightarrow$ PREDICTION)

Pattern	Count
Waiting $\rightarrow$ Executing	20
Stalled $\rightarrow$ Executing	5
Abandoned $\rightarrow$ Stalled	4

VI. EVALUATION

A. Synthetic Corpus

Because no public benchmark exists for agent intent-state labelling, we constructed a reproducible synthetic corpus of 100 traces: 20 each for Completed, Recovering, Waiting, Stalled, and Abandoned behaviour. Generators vary time-stamps, action diversity, failure counts, repetition streaks, and runtime while preserving class semantics. The generator and evaluation scripts are included in the repository (backend/scripts/).

B. Rule-Engine Results

Table I reports per-class precision, recall, and F1 for the rule-based analyzer. Overall accuracy is 71% with macro-F1 of 0.732.

C. Confusion Patterns

Table II summarises the main confusion patterns. All 20 Waiting traces were classified as Executing because polling behaviour is not encoded in the current rule set. Five Stalled traces were classified as Executing when repetition did not reduce the progress score below the stall threshold. Four Abandoned traces were classified as Stalled when repetition dominated over runtime signals.

These errors motivate the LLM behavioral analyzer: they occur where semantics matter more than thresholded statistics. A full LLM evaluation on the same corpus requires API access and is left as ongoing work; the dual-analyzer dashboard is designed to surface such cases interactively.

D. Preset Scenario Validation

Six hand-crafted dashboard scenarios (Healthy, Recovering, Looping, API Failure, Waiting, Abandoned) provide qualitative validation and demonstration paths for reviewers. They complement but do not replace the quantitative synthetic evaluation.

VII. LIMITATIONS

- Rule thresholds are hand-tuned, not optimised on labelled data.
- The synthetic corpus approximates real agent behaviour but is not drawn from production systems.
- Waiting behaviour is not representable in the current rule vocabulary.
- LLM evaluation at scale was not completed in this study due to API dependence.
- No human inter-rater study has validated the state labels.

VIII. CONCLUSION

LLMSheriff explores hybrid intent-state inference for autonomous AI agents. A rule engine provides fast, explainable baselines; an LLM behavioral analyzer targets semantically ambiguous traces; and a behavior interpretation layer treats agreement as confidence and disagreement as diagnostic signal. Evaluation on 100 synthetic traces shows strong performance on clear cases (Completed and Recovering: F1 = 1.00) and systematic weakness on Waiting behaviour, supporting the hybrid design motivation. Future work includes labelled real-world traces, calibrated threshold learning, and large-scale LLM comparison.

REFERENCES

- [1] J. Horkoff et al., “Goal-oriented requirements engineering: An extended systematic mapping study,” *Requirements Engineering*, vol. 21, no. 2, pp. 133–160, 2016.
- [2] A. Dardenne, A. van Lamsweerde, and S. Fickas, “Goal-directed requirements acquisition,” *Science of Computer Programming*, vol. 20, nos. 1–2, pp. 3–50, 1993.
- [3] S. Carberry, “Techniques for plan recognition,” *User Modeling and User-Adapted Interaction*, vol. 11, nos. 1–2, pp. 31–48, 2001.
- [4] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [5] OpenTelemetry Authors, “OpenTelemetry specification v1.26,” <https://opentelemetry.io/docs/specs/otel/>, 2024.
- [6] LangChain, “LangSmith,” <https://www.langchain.com/langsmith>, 2024.
- [7] Langfuse, “Open-source LLM observability,” <https://langfuse.com>, 2024.
- [8] NVIDIA, “Nemotron-3-Ultra-550B,” <https://build.nvidia.com/>, 2025.